

MANUAL DE ARQUITECTURA DE SOFTWARE

Directrices de Estructuración, Modularidad y Reglas de Oro desde Cero

1. Filosofía de Diseño: Arquitectura Modular Limpia

En aplicaciones interactivas de alta demanda computacional, como el modelado geométrico, procesamiento de planos de corte y renderizado 2D en tiempo real (ej. DecoToolkit), una estructura desordenada provoca degradación del rendimiento, fugas de memoria y bloqueos en el desarrollo. La Arquitectura Limpia (Clean Architecture) propone aislar la lógica técnica de negocio de los detalles de infraestructura física y la capa visual.

Capa de Presentación (UI Pura & Declarativa):

Encargada únicamente de pintar la interfaz gráfica en pantalla a partir de propiedades (props) y reaccionar a interacciones básicas. No contiene estados complejos del negocio, llamadas a APIs ni persistencia. Esta capa es 100% intercambiable (puedes cambiar de framework de UI sin alterar la lógica de cálculo).

Capa de Dominio (Controladores y Estado en Custom Hooks):

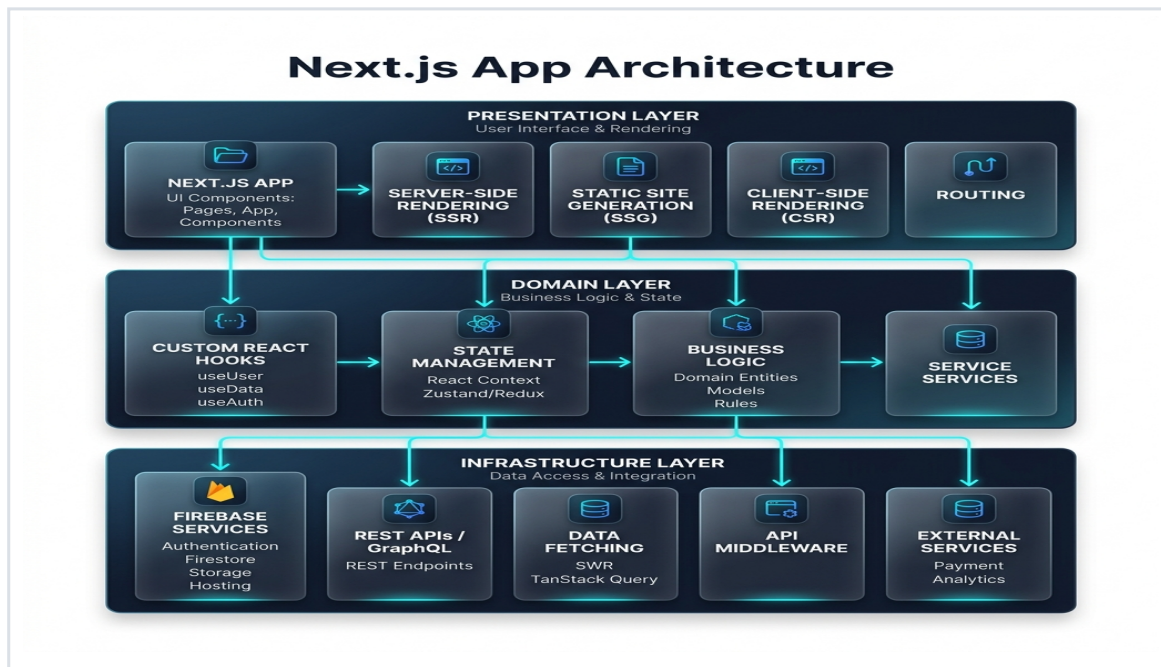
Constituye el núcleo pensante de la aplicación. Aquí residen las máquinas de estado locales, cálculos geométricos, validaciones, control de historial (Undo/Redo) y la captura dinámica de eventos (como el arrastre sobre el canvas o atajos rápidos del teclado). Se implementa mediante ganchos reactivos especializados de TypeScript.

Capa de Infraestructura (Servicios, SDKs y APIs):

Define cómo se envían, guardan y recuperan los datos del mundo exterior. Encapsula los adaptadores de base de datos (como Firestore o PostgreSQL), pasarelas de pago (como TiloPay), y clientes HTTP (como Axios). Los componentes visuales nunca invocan a esta capa de forma directa, protegiendo la integridad transaccional.

2. Diagrama Técnico de Flujo Unidireccional

El siguiente diagrama ilustra la interacción ordenada de las capas. El flujo viaja de forma descendente y unidireccional desde el usuario hacia los hooks de control, y luego a la persistencia externa asíncrona:



2.1 El Flujo de las 3 Capas en palabras sencillas:

- 1. La Presentación (La Piel): Es lo que el usuario ve y toca (botones, pantallas, canvas). Su único trabajo es detectar acciones (ej. "el usuario hizo clic en Guardar Muro") y enviarlas inmediatamente a su cerebro.
- 2. El Dominio (El Cerebro): Son los custom hooks. Reciben las señales de la UI, piensan, hacen cálculos matemáticos, verifican reglas de negocio (ej. "validar que la pieza no exceda el tamaño de la lámina") y actualizan el estado visual.
- 3. La Infraestructura (Las Manos): Es el mensajero o adaptador de base de datos. Cuando el Cerebro decide que todo está correcto, le entrega los datos limpios a las Manos para que los guarde en la base de datos externa (Firestore).
- 4. Flujo de Retorno: Una vez que Firestore confirma el guardado, las Manos le avisan al Cerebro (Hook) y este le ordena a la Piel (UI) que muestre un mensaje de confirmación o actualice el lienzo de forma fluida.

3. Las Cuatro Reglas de Oro del Desarrollo Profesional

Para garantizar la mantenibilidad a largo plazo del código y asegurar un desarrollo ágil y escalable en entornos profesionales, todo el equipo debe seguir sin excepciones las siguientes cuatro reglas de oro de estructuración:

1. Separación Estricta de Responsabilidades (SRP)

Los archivos visuales de React/Next.js no deben procesar lógica de base de datos ni albergar fórmulas matemáticas complejas. Las pantallas actúan únicamente como unificadores y se alimentan de callbacks y estados provistos por hooks y servicios puros, evitando re-renders pesados y manteniendo la interfaz ágil y fluida.

2. Principio de Co-locación y Alcance Local

Reduce el acoplamiento global guardando componentes, hooks y tipos utilitarios en carpetas privadas locales (`_components`, `_hooks`, `_utils`) dentro de la ruta de la pantalla que los consume. Sube un recurso a la carpeta global compartida únicamente cuando sea requerido activamente en dos o más módulos independientes.

3. Formularios Modulares y Validaciones Aisladas

Los formularios gigantes y monolíticos dificultan el testing y la modularidad de la UI. Divide las interfaces de captura de datos en sub-componentes atómicos orientados a tareas específicas. Mantén las reglas y esquemas de validación (esquemas de Zod) en archivos dedicados, permitiendo su reuso en validaciones de cliente y servidor.

4. Integridad Absoluta de Datos y Aislamiento del SDK

Desacopla por completo la base de datos (Firestore, Supabase, etc.) inyectándola exclusivamente a través de servicios o hooks de mutación específicos. Implementa bloques try/catch detallados para manejar fallos de red, sanitiza payloads antes de escribir para erradicar undefineds, y utiliza transacciones atómicas (`writeBatch`) al modificar entidades vinculadas.